ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

502094 - Software Deployment, Operations And Maintenance

# Module 11: Infrastructure as Code (IaC)

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

# Introduction to Infrastructure as Code (IaC)

- **Infrastructure as Code (IaC)**: managing infrastructure using configuration files instead of manual clicks
- Treat servers, networks, and cloud resources as **code** in a Git repository
- Enables:
  - Reproducible environments
  - Automated provisioning and updates
  - Collaboration and code review for infrastructure
- Central question: *Can we set up and tear down an entire environment with a single command?*

# Why We Need IaC

- Manual provisioning issues:
  - Clicks in web consoles are hard to reproduce
  - “It works on my AWS account but not yours”
  - No history of who changed what
- IaC benefits:
  - Version control for infrastructure
  - Rollback to known-good configurations
  - Consistent environments across dev / staging / production

# Manual vs Automated Provisioning

- **Manual provisioning**:
  - Create EC2 instance via AWS Console
  - Manually open ports
  - Copy/paste IPs to share with team
- **Automated provisioning** with IaC:
  - Write `.tf` files (Terraform)
  - Review changes in pull requests
  - Run `terraform apply` to create/update resources
- Result:
  - Faster, safer, more repeatable workflows

# Declarative vs Imperative Models

- **Imperative**:
  - "Do this, then that"

```
aws ec2 run-instances ...
aws ec2 authorize-security-group-ingress ...
```

- **Declarative**:
  - "This is the state I want"

```
resource "aws_instance" "web" {
  ami           = "ami-1234567890abcdef0"
  instance_type = "t3.micro"
}
```

- Terraform: Reads desired state → decides *how* to reach it

# Idempotency and Reproducibility

- **Idempotency**:
  - Running `terraform apply` multiple times leads to the **same result**
  - Avoids double-creating resources
- **Reproducibility**:
  - Same code + same variables ⇒ same infrastructure
- For DevOps:
  - Safer iteration
  - Easy environment rebuild after failures
  - Supports disaster recovery scenarios

# Why Terraform?

- Open-source **cloud-agnostic** IaC tool
- Single language (HCL) to manage:
  - AWS, Azure, GCP
  - DNS, GitHub, Kubernetes, Cloudflare, etc.
- Fully declarative:
  - Compute plan → confirm → apply
- Great fit for teaching and real-world DevOps teams

# Terraform Provider Ecosystem

- **Providers** are plugins to talk to different APIs:
  - `aws`, `azurerm`, `google`, `kubernetes`, `helm`, `github`, `cloudflare`
- Each provider defines:
  - Which **resources** you can create
  - Which **data sources** you can read

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

# Terraform Plan → Apply Workflow

- Typical lifecycle:
  i. Write or change `.tf` files
  ii. `terraform init` – download providers and initialize backend
  iii. `terraform plan` – preview changes
  iv. `terraform apply` – apply the plan
- Key idea:
  - Always **review** the plan before applying
  - Prevents unexpected infrastructure changes

# Terraform State Management Advantages

- Terraform keeps a **state file** to remember:
  - Resource IDs
  - Dependencies
  - Metadata
- Benefits:
  - Detects **drift** between code and real cloud
  - Enables **incremental** updates instead of full recreation
- Advanced:
  - Remote state storage and locking for teams

# Terraform vs Other IaC Tools

- **CloudFormation**:
  - AWS-only, YAML/JSON templates
- **Pulumi**:
  - Use TypeScript/Python/Go as IaC language
- **Ansible**:
  - Great for configuration management and provisioning
  - Less ideal for tracking long-lived cloud resources
- Terraform strengths:
  - Strong multi-cloud story
  - Simple declarative language
  - Powerful ecosystem and tooling

# Installing Terraform & Key Terminology

- Install options:
  - Homebrew (macOS): `brew install terraform`
  - Direct binary download from terraform.io
  - Package managers (choco, apt, etc.)
- Once installed:

```
terraform -v
```

- Learn core terms:
  - Provider, resource, data source, variables, outputs, modules, state, backend

# Terraform Provider

- **Provider**:
  - Responsible for managing a specific API
- Example:

```
provider "aws" {
  region  = "ap-southeast-1"
  profile = "tdtu-devops"
}
```

- Considerations:
  - Region selection (e.g., `ap-southeast-1` for Singapore)
  - Authentication via AWS profiles or environment variables

# Terraform Resource

- **Resource**:
  - A piece of infrastructure managed by Terraform
- Example EC2 instance:

```
resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t3.micro"

  tags = {
    Name = "tdtu-web-01"
  }
}
```

- Terraform:
  - Creates, updates, or deletes the resource based on code changes

# Data Sources, Variables, Outputs

- **Data source**:
  - Read information from cloud (e.g., latest AMI)
- **Variable**:
  - Parameterize configurations
- **Output**:
  - Expose useful information after apply (e.g., public IP)
- Example variable:

```
variable "instance_type" {
  type    = string
  default = "t3.micro"
}
```

# Modules, State, Backend (Concepts)

- **Module**:
  - Reusable Terraform configuration
  - Think “function” or “library” for infrastructure
- **State**:
  - Saved record of deployed resources
- **Backend**:
  - Where the state file is stored
  - Local file vs remote backend (S3, Terraform Cloud)

# Terraform State – The Most Critical Concept

- `terraform.tfstate`:
  - JSON file storing the mapping between Terraform resources and real cloud resources
- Without state:
  - Terraform would not know what already exists
- With state:
  - Terraform can compute a **diff** between desired config and actual infrastructure

# Why Terraform Keeps State

- State tracks:
  - Resource IDs (e.g., EC2 instance IDs)
  - Dependencies (e.g., security group used by instance)
- Enables:
  - Precise updates: only change what is necessary
  - `terraform plan` to show exactly what will change
- Risk:
  - If state is corrupted or lost, Terraform loses track of resources

# Drift Detection

- **Drift**:
  - Real infrastructure changes that are not in Terraform code
  - Example:
    - Someone manually opens port 22 to 0.0.0.0/0 in AWS Console
- Terraform:
  - `terraform plan` compares:
    - State file
    - Cloud API
    - Code
  - Shows drift and suggests changes to reconcile

# Local vs Remote State

- **Local state**:
  - Default: `terraform.tfstate` stored on your machine
  - Good for learning and small projects
- Risks:
  - Easy to lose state
  - Hard for teams to collaborate
- **Remote state**:
  - Store in S3, Terraform Cloud, etc.
  - Enables team workflows and locking

# Remote State: S3 + DynamoDB (Conceptual)

- Typical production setup:
  - State stored in S3 bucket
  - Locking handled by DynamoDB
- Example backend block:

```
terraform {
  backend "s3" {
    bucket         = "tdtu-devops-terraform-state"
    key            = "envs/prod/network/terraform.tfstate"
    region         = "ap-southeast-1"
    dynamodb_table = "tdtu-terraform-locks"
  }
}
```

- Benefit: Single source of truth shared across team

# Variables & .tfvars Files

- **Variables** make Terraform flexible:
  - Different environments, same module
- Simple declaration:

```
variable "env" {
  type = string
}
```

- Usage:

```
tags = {
  Environment = var.env
}
```

# Required vs Optional Variables

- Required:
  - No `default` value
  - Must be provided by:
    - CLI `-var`
    - `.tfvars` file
- Optional:
  - Have `default` value
- Example:

```
variable "instance_type" {
  type    = string
  default = "t3.micro"
}
```

# Using terraform.tfvars and auto.tfvars

- `terraform.tfvars`:
  - Automatically loaded if present
- `*.auto.tfvars`:
  - Any file ending with `.auto.tfvars` is also auto-loaded
- Example `terraform.tfvars`:

```
env           = "dev"
instance_type = "t3.small"
```

- Good practice:
  - Keep **non-sensitive** environment config here

# Overriding Variables via CLI

- Provide variables at runtime:

```
terraform apply -var="env=staging" -var="instance_type=t3.medium"
```

- Or via a specific file:

```
terraform apply -var-file="staging.tfvars"
```

- Priority:
  - CLI vars override `.tfvars` and defaults

# Sensitive Variables

- Use `sensitive = true` for secrets:

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

- Terraform:
  - Avoids printing sensitive values in plan/apply
- Still:
  - Do not commit actual secrets to Git
  - Inject via CI/CD or secure variable stores

# Best Practices for Variables

- Separate:
  - Code (`.tf`)
  - Config (`.tfvars`)
- Keep secrets out of Git:
  - Use environment variables or CI/CD secrets
- Use consistent naming:
  - `env`, `region`, `project`, `owner`
- Use type constraints:
  - `type = string`, `list(string)`, `map(string)` to catch mistakes early

# Basic Terraform Workflows & CLI Commands

- Terraform core commands:
  - `terraform init`
  - `terraform fmt`
  - `terraform validate`
  - `terraform plan`
  - `terraform apply`
  - `terraform destroy`
  - `terraform output`
- Workflow:
  - Edit code → fmt → validate → plan → apply

# terraform init

- Initializes working directory:

```
terraform init
```

- Responsibilities:
  - Download required providers
  - Initialize backend (if configured)
- Must run:
  - After changing providers
  - After cloning repo for the first time

# terraform fmt and validate

- `terraform fmt`:
  - Formats `.tf` files
  - Enforces standard style

```
terraform fmt
```

- `terraform validate`:
  - Checks syntax and basic configuration validity

```
terraform validate
```

- Use both:
  - Before committing changes
  - In CI pipelines

# terraform plan

- Previews changes:

```
terraform plan
```

- Shows:
  - Resources to create (`+`)
  - To update (`~`)
  - To destroy (`-`)
- Example output:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

- Always review the plan for safety

# terraform apply

- Applies the plan:

```
terraform apply
```

- Interactively:
  - Shows plan and asks for `yes` confirmation
- Non-interactive (CI/CD):

```
terraform apply -auto-approve
```

- After apply:
  - State file is updated with new resource IDs

# terraform destroy

- Destroys all managed resources:

```
terraform destroy
```

- Use with caution:
  - Double-check which environment you are in
- Safe patterns:
  - Separate state for dev/staging/prod
  - Protect critical resources via `prevent_destroy` where needed

# terraform output and State Basics

- `terraform output`:
  - Shows output values defined in configuration

```
terraform output
terraform output web_public_ip
```

- State basics:
  - `terraform.tfstate` maps logical names to real resource IDs
- Avoid:
  - Editing `terraform.tfstate` manually unless absolutely necessary

# Outputs Feeding Into CI/CD Pipelines

- **Outputs** make Terraform results consumable by:
  - Shell scripts
  - GitHub Actions
  - CD tools
- Example output:

```
output "web_public_ip" {
  value = aws_instance.web.public_ip
}
```

# Using Outputs in Shell Scripts

- Capture output:

```
WEB_IP=$(terraform output -raw web_public_ip)
echo "Web server IP is: $WEB_IP"
curl "http://$WEB_IP"
```

- Scenario:
  - After provisioning EC2 instance, run deployment script using the IP

# Using Outputs in GitHub Actions

- Example job step:

```yaml
- name: Get Terraform outputs
  id: tf
  run: |
    cd infra
    terraform output -raw web_public_ip > web_ip.txt
  shell: bash
```

- Later steps can:
  - Read `web_ip.txt`
  - Use it to configure deployment of a Node.js or Spring Boot app

# Working With Terraform & AWS (Hands-On Focus)

- We will:
  - Set up AWS credentials
  - Generate SSH keys
  - Create key pairs and security groups
  - Provision EC2 instances
  - Expose outputs (IP addresses) for later use
- Focus on:
  - **AWS EC2**
  - **Security Groups**
  - **Key pairs**

# Setting Up AWS Credentials

- Install AWS CLI:

```
sudo apt install awscli    # Ubuntu
brew install awscli        # macOS
```

- Configure profile:

```
aws configure --profile tdtu-devops
```

- Provide:
  - Access key ID
  - Secret access key
  - Default region (e.g., `ap-southeast-1`)
  - Output format (e.g., `json`)

# Using AWS Profiles in Terraform

- Terraform provider referencing profile:

```
provider "aws" {
  region  = "ap-southeast-1"
  profile = "tdtu-devops"
}
```

- Alternative:
  - Use environment variables:

```
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export AWS_DEFAULT_REGION=ap-southeast-1
```

# Generating SSH Key Pairs (ssh-keygen)

- On your local machine:

```
ssh-keygen -t rsa \
  -b 4096 \
  -C "devops-student@tdtu.edu.vn" \
  -f ~/.ssh/tdtu-devops-key
```

- Files created:
  - Private key: `~/.ssh/tdtu-devops-key`
  - Public key: `~/.ssh/tdtu-devops-key.pub`
- Principle:
  - **Never** commit private keys to Git
  - Protect private keys with proper permissions

# Uploading SSH Key via aws_key_pair

- Terraform resource:

```
resource "aws_key_pair" "tdtu_key" {
  key_name   = "tdtu-devops-key"
  public_key = file("~/.ssh/tdtu-devops-key.pub")
}
```

- Usage:
  - Attach `aws_key_pair.tdtu_key.key_name` to EC2 instances
- Benefit:
  - Managed entirely via code, easy to recreate

# Creating Security Groups

```
resource "aws_security_group" "web_sg" {
  name        = "tdtu-web-sg"
  description = "Allow SSH and HTTP"
  vpc_id      = data.aws_vpc.default.id
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["203.113.0.0/16"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

# Security Groups & Principle of Least Privilege

- **Least privilege**:
  - Open only the ports and IP ranges you truly need
- Examples:
  - SSH (22):
    - Restrict to university IP range or VPN
  - Application ports (3000, 8080):
    - Expose only through load balancer if possible
- Logs:
  - Keep track of security group changes via Git history

# Dynamic Blocks for Multiple Ports

```
variable "app_ports" {
  type    = list(number)
  default = [3000, 8080]
}
resource "aws_security_group" "app_sg" {
  name   = "tdtu-app-sg"
  vpc_id = data.aws_vpc.default.id
  dynamic "ingress" {
    for_each = var.app_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

# Creating EC2 Instances

- Basic EC2 instance:

```
resource "aws_instance" "web" {
  ami                    = "ami-0abcdef1234567890"
  instance_type          = "t3.micro"
  key_name               = aws_key_pair.tdtu_key.key_name
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  tags = {
    Name        = "tdtu-web-01"
    Environment = var.env
    Owner       = "DevOps Class"
  }
}
```

- For Ubuntu 22.04: Use appropriate region-specific AMI ID

# Instance Tags and Naming Conventions

- Use consistent naming:
  - `project-env-role-index`
  - Example: `booking-dev-api-01`
- Tags:

```
tags = {
  Name        = "booking-dev-api-01"
  Environment = "dev"
  Project     = "booking-service"
  Owner       = "mvm"
}
```

- Benefits:
  - Easier billing analysis
  - Simpler filtering and automation

# Showing Public & Private IP Addresses

- Define outputs:

```
output "web_public_ip" {
  value = aws_instance.web.public_ip
}

output "web_private_ip" {
  value = aws_instance.web.private_ip
}
```

- After `terraform apply`:

```
terraform output web_public_ip
terraform output web_private_ip
```

# Verify EC2 Instance via AWS Console

- After provisioning:
  - Open AWS Console → EC2 → Instances
  - Check:
    - Instance state: running
    - Public IP matches Terraform output
    - Security group attached correctly
- Use SSH to connect:

```
ssh -i ~/.ssh/tdtu-devops-key ubuntu@<web_public_ip>
```

# Destroying Infrastructure Safely

- Destroy all resources:

```
terraform destroy
```

- Best practices:
  - Double-check state and workspace
  - Avoid running as root
- For teaching:
  - Encourage students to **destroy dev resources** after labs to save cost

# Selective Destruction & Protection

- Destroy specific resource:

```
terraform destroy -target=aws_instance.web
```

- Protect critical resources:

```
resource "aws_s3_bucket" "logs" {
  bucket = "tdtu-prod-logs"

  lifecycle {
    prevent_destroy = true
  }
}
```

- Prevents accidental deletion of critical buckets

# Hands-On Exercises Overview

- **Exercise 1** — Terraform initialization & basic commands
- **Exercise 2** — SSH keys + `aws_key_pair`
- **Exercise 3** — Security groups with multiple ports
- **Exercise 4** — EC2 instance provisioning
- **Exercise 5** — Mini-project: key pair + SG + EC2 + outputs + destroy

# Exercise 1 — Terraform Initialization

- Tasks:
  - Install Terraform
  - Run `terraform init` in a new project folder
  - Create a minimal `main.tf` with only `terraform` and `provider` blocks
- Goal:
  - Understand provider configuration and initialization process

# Exercise 2 — SSH Keys + Key Pair Resource

- Tasks:
  - Generate SSH key pair with `ssh-keygen`
  - Create `aws_key_pair` resource referencing `.pub` file
  - Apply and verify key pair in AWS Console
- Discussion:
  - Why private keys must never be committed
  - How logs help investigate unauthorized SSH access

# Exercise 3 — Security Groups With Multiple Ports

- Tasks:
  - Create security group allowing:
    - SSH (22) from restricted IP range
    - HTTP (80) and HTTPS (443) from anywhere
    - Optional app ports (3000, 8080)
  - Use dynamic blocks for app ports
- Goal:
  - Practice least privilege and reusable patterns

# Exercise 4 — EC2 Instance Provisioning

- Tasks:
  - Use Ubuntu 22.04 AMI
  - Attach key pair and security group
  - Add meaningful tags
  - Output public IP
- Verification:
  - SSH into instance
  - Install Nginx and serve a simple HTML page

# Exercise 5 — Full Mini-Project

- Project:
  - Create a small `infra` module that:
    - Creates key pair
    - Creates security group
    - Creates one EC2 instance
    - Exposes outputs for IP addresses
  - Run:
    - `terraform plan`
    - `terraform apply` and finally run `terraform destroy`
- Outcome: End-to-end experience with IaC lifecycle

# Optional Advanced Topics (If Time Permits)

- Remote state: S3 backend + DynamoDB locking
- Terraform modules:
  - Reusable infrastructure building blocks
- VPC basics:
  - Subnets, route tables, internet gateway
- Tagging strategy:
  - Consistent keys for cost and governance
- Drift detection:
  - Using `terraform plan` in CI
- Importing resources: `terraform import` to bring manual resources under IaC control

# Summary & Next Steps

- You learned:
  - Why IaC matters and how Terraform fits DevOps workflows
  - Core Terraform concepts: providers, resources, variables, state, outputs
  - Practical AWS usage: key pairs, security groups, EC2, outputs, destroy
- Next modules:
  - Combine Terraform with:
    - CI/CD pipelines
    - Kubernetes clusters
    - Monitoring and security tooling
- Key takeaway: Treat infrastructure as **versioned, testable, reviewable code**.